
YOHPAL LIVE RELEASE 1.0F — Final GA Blocker Elimination Code Patch Pack.

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Fri, 10 Jul at 18:34

Below is the **YOHPAL LIVE RELEASE 1.0F — Final GA Blocker Elimination Code Patch Pack**.

The latest implementation report confirms that money/security work is substantially improved, but **4 Critical findings remain open and 0 of 13 High findings are fully closed**. The remaining blockers are concentrated in observability, password recovery, route integrity, environment configuration, discovery wiring, feed recovery, LiveKit resilience, accessibility, duplicate pipelines, and test coverage.

IMPLEMENTATION_STATUS_REPORT.pdf

1. Observability

pubspec.yaml

```
dependencies:  
  firebase_core: ^3.15.0  
  firebase_crashlytics: ^4.3.8  
  firebase_analytics: ^11.4.8
```

Use versions compatible with the project's installed Firebase packages before committing.

lib/core/observability/yohpal_crash_reporter.dart

```
import 'package:firebase_crashlytics/firebase_crashlytics.dart';  
import 'package:flutter/foundation.dart';  
  
abstract interface class YohPalCrashReporter {  
  Future<void> recordError(  
    Object error,  
    StackTrace stackTrace, {  
      bool fatal = false,  
      String? reason,  
    });  
  
  Future<void> recordFlutterError(FlutterErrorDetails details);  
  
  Future<void> setUserId(String? uid);  
  
  Future<void> log(String message);  
}  
  
final class FirebaseYohPalCrashReporter implements YohPalCrashReporter {  
  FirebaseYohPalCrashReporter({  
    FirebaseCrashlytics? crashlytics,  
  }) : _crashlytics = crashlytics ?? FirebaseCrashlytics.instance;  
  
  final FirebaseCrashlytics _crashlytics;  
  
  @override  
  Future<void> recordError(  
    Object error,  
    StackTrace stackTrace, {  
      bool fatal = false,  
      String? reason,  
    }) {  
    return _crashlytics.recordError(  
      error,  
      stackTrace,  
      fatal: fatal,  
      reason: reason,  
    );  
  }  
  
  @override  
  Future<void> recordFlutterError(FlutterErrorDetails details) {  
    return _crashlytics.recordFlutterFatalError(details);  
  }  
  
  @override  
  Future<void> setUserId(String? uid) {  
    return _crashlytics.setUserIdentifier(uid ?? '');  
  }  
}
```

```

}

@override
Future<void> log(String message) {
  return _crashlytics.log(message);
}
}

```

lib/main.dart

```

import 'dart:async';
import 'dart:ui';

import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';

import 'app/app.dart';
import 'core/observability/yohpal_crash_reporter.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  final crashReporter = FirebaseYohPalCrashReporter();

  FlutterError.onError = (details) {
    FlutterError.presentError(details);
    unawaited(crashReporter.recordFlutterError(details));
  };

  PlatformDispatcher.instance.onError = (error, stack) {
    unawaited(
      crashReporter.recordError(
        error,
        stack,
        fatal: true,
        reason: 'PlatformDispatcher uncaught error',
      ),
    );
    return true;
  };

  await runZonedGuarded(
    () async {
      runApp(
        YohPalApp(
          crashReporter: crashReporter,
        ),
      );
    },
    (error, stack) {
      unawaited(
        crashReporter.recordError(
          error,
          stack,
          fatal: true,
          reason: 'Uncaught zoned error',
        ),
      );
    },
  );
}

```

Temporary test action

```

Future<void> triggerCrashlyticsVerification() async {
  await FirebaseCrashlytics.instance.log(
    'Release 1.0F verification crash',
  );

  FirebaseCrashlytics.instance.crash();
}

```

Keep this behind a debug/admin flag and remove the visible trigger after proof is captured.

2. Environment Configuration

The report confirms production endpoints remain hardcoded.

IMPLEMENTATION_STATUS_REPORT.pdf

lib/core/config/yohpal_environment.dart

```
enum YohPalEnvironmentName {
  development,
  staging,
  production,
}

final class YohPalEnvironment {
  const YohPalEnvironment({
    required this.name,
    required this.functionsBaseUrl,
    required this.liveKitUrl,
    required this.walletUrl,
    required this.adsUrl,
    required this.subscriptionsUrl,
  });

  final YohPalEnvironmentName name;
  final Uri functionsBaseUrl;
  final Uri liveKitUrl;
  final Uri walletUrl;
  final Uri adsUrl;
  final Uri subscriptionsUrl;

  factory YohPalEnvironment.fromDefines() {
    final environmentName = const String.fromEnvironment(
      'YOHPAL_ENV',
      defaultValue: 'development',
    );

    return YohPalEnvironment(
      name: switch (environmentName) {
        'production' => YohPalEnvironmentName.production,
        'staging' => YohPalEnvironmentName.staging,
        _ => YohPalEnvironmentName.development,
      },
      functionsBaseUrl: _requiredUri(
        'YOHPAL_FUNCTIONS_BASE_URL',
        const String.fromEnvironment('YOHPAL_FUNCTIONS_BASE_URL'),
      ),
      liveKitUrl: _requiredUri(
        'YOHPAL_LIVEKIT_URL',
        const String.fromEnvironment('YOHPAL_LIVEKIT_URL'),
      ),
      walletUrl: _requiredUri(
        'YOHPAL_WALLET_URL',
        const String.fromEnvironment('YOHPAL_WALLET_URL'),
      ),
      adsUrl: _requiredUri(
        'YOHPAL_ADS_URL',
        const String.fromEnvironment('YOHPAL_ADS_URL'),
      ),
      subscriptionsUrl: _requiredUri(
        'YOHPAL_SUBSCRIPTIONS_URL',
        const String.fromEnvironment('YOHPAL_SUBSCRIPTIONS_URL'),
      ),
    );
  }

  static Uri _requiredUri(String key, String value) {
    final uri = Uri.tryParse(value);

    if (value.isEmpty || uri == null || !uri.hasScheme) {
      throw StateError('Missing or invalid environment value: $key');
    }

    return uri;
  }
}
```

Run configuration

```
flutter run \
--dart-define=YOHPAL_ENV=staging \
--dart-define=YOHPAL_FUNCTIONS_BASE_URL=https://staging-functions.example.com \
--dart-define=YOHPAL_LIVEKIT_URL=wss://staging-livekit.example.com \
--dart-define=YOHPAL_WALLET_URL=https://staging-wallet.example.com \
--dart-define=YOHPAL_ADS_URL=https://staging-ads.example.com \
--dart-define=YOHPAL_SUBSCRIPTIONS_URL=https://staging-subscriptions.example.com
```

3. Authentication Completion

lib/features/auth/data/auth_error_mapper.dart

```
import 'package:firebase_auth/firebase_auth.dart';

final class AuthErrorMapper {
  const AuthErrorMapper._();

  static String message(Object error) {
    if (error is FirebaseAuthException) {
      return switch (error.code) {
        'invalid-email' => 'Enter a valid email address.',
        'invalid-credential' ||
        'wrong-password' =>
          'The email or password is incorrect.',
        'user-not-found' => 'No account exists for that email.',
        'email-already-in-use' =>
          'An account already exists for that email.',
        'weak-password' =>
          'Use a stronger password with at least eight characters.',
        'too-many-requests' =>
          'Too many attempts. Wait briefly and try again.',
        'network-request-failed' =>
          'Check your internet connection and try again.',
        'user-disabled' =>
          'This account has been disabled. Contact support.',
        _ => 'We could not complete that request. Please try again.',
      };
    }

    return 'Something went wrong. Please try again.';
  }
}
```

lib/features/auth/data/password_reset_service.dart

```
import 'package:firebase_auth/firebase_auth.dart';

final class PasswordResetService {
  PasswordResetService({
    FirebaseAuth? auth,
  }) : _auth = auth ?? FirebaseAuth.instance;

  final FirebaseAuth _auth;

  Future<void> sendResetEmail(String rawEmail) async {
    final email = rawEmail.trim();

    if (!isValidEmail(email)) {
      throw const FormatException('Enter a valid email address.');
```

lib/features/auth/screens/forgot_password_screen.dart

```
import 'package:flutter/material.dart';

import '../data/auth_error_mapper.dart';
import '../data/password_reset_service.dart';

class ForgotPasswordScreen extends StatefulWidget {
  const ForgotPasswordScreen({
    super.key,
    this.service,
  });

  static const routeName = '/forgot-password';

  final PasswordResetService? service;
```

```

@override
State<ForgotPasswordScreen> createState() =>
  _ForgotPasswordScreenState();
}

class _ForgotPasswordScreenState extends State<ForgotPasswordScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();

  late final PasswordResetService _service =
    widget.service ?? PasswordResetService();

  bool _submitting = false;
  String? _error;
  bool _sent = false;

  @override
  void dispose() {
    _emailController.dispose();
    super.dispose();
  }

  Future<void> _submit() async {
    if (!_formKey.currentState!.validate()) return;

    setState(() {
      _submitting = true;
      _error = null;
    });

    try {
      await _service.sendResetEmail(_emailController.text);

      if (!mounted) return;

      setState(() => _sent = true);
    } catch (error) {
      if (!mounted) return;

      setState(() {
        _error = error is FormatException
          ? error.message
          : AuthErrorMapper.message(error);
      });
    } finally {
      if (mounted) {
        setState(() => _submitting = false);
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Reset password')),
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: _sent
            ? const Center(
                child: Text(
                  'Check your email for password reset instructions.',
                  textAlign: TextAlign.center,
                ),
              ),
            : Form(
                key: _formKey,
                child: Column(
                  children: [
                    TextFormField(
                      controller: _emailController,
                      keyboardType: TextInputType.emailAddress,
                      autofillHints: const [AutofillHints.email],
                      decoration: const InputDecoration(
                        labelText: 'Email address',
                      ),
                      validator: (value) {
                        final email = value?.trim() ?? '';

                        if (email.isEmpty) {
                          return 'Email is required.';
                        }

                        if (!RegExp(
                          r'^[@\s]+\.[^\s]+\.[^\s]+\$',
                        ).hasMatch(email)) {

```



```

final class AppRoutes {
  const AppRoutes._();

  static const search = '/search';
  static const liveDiscovery = '/live-discovery';
  static const walletStatus = '/wallet-status';
  static const forgotPassword = '/forgot-password';
  static const creatorAnalytics = '/creator-profile/analytics';
}

```

Discover hub actions

```

Row(
  children: [
    Semantics(
      label: 'Search YohPal Live',
      button: true,
      child: IconButton(
        tooltip: 'Search',
        icon: const Icon(Icons.search),
        onPressed: () {
          Navigator.pushNamed(context, AppRoutes.search);
        },
      ),
    ),
    Semantics(
      label: 'Discover live streams',
      button: true,
      child: IconButton(
        tooltip: 'Live now',
        icon: const Icon(Icons.live_tv),
        onPressed: () {
          Navigator.pushNamed(
            context,
            AppRoutes.liveDiscovery,
          );
        },
      ),
    ),
  ],
)

```

Route registration

```

routes: {
  AppRoutes.search: (_) => const SearchScreen(),
  AppRoutes.liveDiscovery: (_) => const LiveDiscoveryScreen(),
  AppRoutes.walletStatus: (_) => const WalletStatusScreen(),
  AppRoutes.forgotPassword: (_) => const ForgotPasswordScreen(),
  AppRoutes.creatorAnalytics: (_) =>
    const CreatorAnalyticsScreen(),
},
onUnknownRoute: (settings) {
  return MaterialPageRoute<void>(
    builder: (_) => UnknownRouteScreen(
      attemptedRoute: settings.name,
    ),
  );
},

```

lib/core/routing/unknown_route_screen.dart

```

import 'package:flutter/material.dart';

class UnknownRouteScreen extends StatelessWidget {
  const UnknownRouteScreen({
    super.key,
    this.attemptedRoute,
  });

  final String? attemptedRoute;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Page unavailable')),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              const Icon(Icons.link_off, size: 48),

```

```

        const SizedBox(height: 16),
        const Text(
          'This destination is unavailable.',
          textAlign: TextAlign.center,
        ),
        if (attemptedRoute != null)
          Text(
            attemptedRoute!,
            textAlign: TextAlign.center,
          ),
        const SizedBox(height: 24),
        FilledButton(
          onPressed: () {
            Navigator.of(context)
              .popUntil((route) => route.isFirst);
          },
          child: const Text('Return home'),
        ),
      ],
    ),
  ),
);
}
}

```

5. Feed Recovery and Pagination

lib/features/video_feed/data/video_feed_page.dart

```

import 'package:cloud_firestore/cloud_firestore.dart';

final class VideoFeedPage<T> {
  const VideoFeedPage({
    required this.items,
    required this.lastDocument,
    required this.hasMore,
  });

  final List<T> items;
  final DocumentSnapshot<Map<String, dynamic>>? lastDocument;
  final bool hasMore;
}

```

Repository pagination

```

Future<VideoFeedPage<VideoFeedItem>> loadPage({
  DocumentSnapshot<Map<String, dynamic>>? after,
  int pageSize = 25,
}) async {
  Query<Map<String, dynamic>> query = firestore
    .collection('videos')
    .where('visibility', isEqualTo: 'public')
    .where('status', isEqualTo: 'ready')
    .orderBy('createdAt', descending: true)
    .limit(pageSize);

  if (after != null) {
    query = query.startAfterDocument(after);
  }

  final snapshot = await query.get();

  final items = snapshot.docs
    .map(VideoFeedItem.fromDocument)
    .toList(growable: false);

  return VideoFeedPage(
    items: items,
    lastDocument:
      snapshot.docs.isEmpty ? after : snapshot.docs.last,
    hasMore: snapshot.docs.length == pageSize,
  );
}

```

Error state

```

class FeedRecoveryView extends StatelessWidget {
  const FeedRecoveryView({
    super.key,
    required this.onRetry,
  });
}

```



```

});

final Future<void> Function() onRetry;

@override
Widget build(BuildContext context) {
  return Center(
    child: Semantics(
      label: 'Video feed failed to load',
      liveRegion: true,
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Icon(Icons.cloud_off, size: 48),
          const SizedBox(height: 12),
          const Text('We could not load your feed.'),
          const SizedBox(height: 16),
          FilledButton.icon(
            onPressed: onRetry,
            icon: const Icon(Icons.refresh),
            label: const Text('Try again'),
          ),
        ],
      ),
    ),
  );
}

```

6. Mute, Double-Tap and Accessibility

lib/features/video_feed/widgets/video_interaction_surface.dart

```

import 'package:flutter/material.dart';

class VideoInteractionSurface extends StatelessWidget {
  const VideoInteractionSurface({
    super.key,
    required this.child,
    required this.creatorName,
    required this.caption,
    required this.isMuted,
    required this.onTogglePlayback,
    required this.onLike,
    required this.onToggleMute,
  });

  final Widget child;
  final String creatorName;
  final String caption;
  final bool isMuted;
  final VoidCallback onTogglePlayback;
  final VoidCallback onLike;
  final VoidCallback onToggleMute;

  @override
  Widget build(BuildContext context) {
    return Semantics(
      container: true,
      label: 'Video by $creatorName. $caption',
      hint:
        'Single tap to play or pause. Double tap to like.',
      child: GestureDetector(
        behavior: HitTestBehavior.opaque,
        onTap: onTogglePlayback,
        onDoubleTap: onLike,
        child: Stack(
          fit: StackFit.expand,
          children: [
            child,
            Positioned(
              top: 16,
              right: 16,
              child: Semantics(
                button: true,
                label: isMuted ? 'Unmute video' : 'Mute video',
                child: IconButton.filledTonal(
                  tooltip: isMuted ? 'Unmute' : 'Mute',
                  onPressed: onToggleMute,
                  icon: Icon(
                    isMuted
                      ? Icons.volume_off
                      : Icons.volume_up,

```


Wire it to the room's disconnect/reconnect events and show a visible retry banner rather than silently hanging.

8. Duplicate Pipeline Removal

The latest report confirms lib/core/video/ and lib/core/video_feed/ remain present and unreferenced.

IMPLEMENTATION_STATUS_REPORT.pdf

scripts/remove_duplicate_video_pipelines.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "$0")/.." && pwd)"
cd "$ROOT_DIR"

echo "Checking references before deletion..."

if grep -R \
  --exclude-dir=.dart_tool \
  --exclude-dir=build \
  --exclude-dir=.git \
  "core/video/" lib test; then
  echo "References to lib/core/video remain. Aborting."
  exit 1
fi

if grep -R \
  --exclude-dir=.dart_tool \
  --exclude-dir=build \
  --exclude-dir=.git \
  "core/video_feed/" lib test; then
  echo "References to lib/core/video_feed remain. Aborting."
  exit 1
fi

rm -rf lib/core/video
rm -rf lib/core/video_feed

dart format lib test
flutter analyze
flutter test

echo "Duplicate video pipelines removed successfully."
```

Do not delete files solely because the audit says they are unused; run the reference guard against the developer's current HEAD.

9. Automated Route Integrity Gate

tool/check_routes.dart

```
import 'dart:io';

void main() {
  final libDirectory = Directory('lib');

  if (!libDirectory.existsSync()) {
    stderr.writeln('lib directory not found.');
```

```
    exitCode = 2;
    return;
  }

  final dartFiles = libDirectory
    .listSync(recursive: true)
    .whereType<File>()
    .where((file) => file.path.endsWith('.dart'))
    .toList();

  final registeredRoutes = <String>{};
  final pushedRoutes = <String>{};

  final routePattern = RegExp(r""[""](/[""]+)["""]);
  final registrationPattern = RegExp(
    r""[""](/[""]+)["""]\s*:""",
  );
  final pushPattern = RegExp(
    r""pushNamed(?:AndRemoveUntil|AndRemoveUntil)?\s*([^\s,]+,\s*[""](/[""]+)["""])",
  );
```

```

for (final file in dartFiles) {
  final content = file.readAsStringSync();

  for (final match in registrationPattern.allMatches(content)) {
    registeredRoutes.add(match.group(1)!);
  }

  for (final match in pushPattern.allMatches(content)) {
    pushedRoutes.add(match.group(1)!);
  }

  if (content.contains('static const routeName')) {
    for (final match in routePattern.allMatches(content)) {
      registeredRoutes.add(match.group(1)!);
    }
  }
}

final missing = pushedRoutes.difference(registeredRoutes);

if (missing.isNotEmpty) {
  stderr.writeln('Unregistered pushed routes:');
  for (final route in missing) {
    stderr.writeln(' - $route');
  }
  exitCode = 1;
  return;
}

stdout.writeln(
  'Route integrity passed: '
  '${pushedRoutes.length} pushed routes verified.',
);
}

```

This provides a starter gate. A stronger long-term implementation should use Dart analyzer AST parsing rather than regular expressions.

10. Test Pack

Password reset test

```

import 'package:flutter_test/flutter_test.dart';
import 'package:yohpal_live/features/auth/data/auth_error_mapper.dart';

void main() {
  test('maps invalid credentials to friendly copy', () {
    expect(
      AuthErrorMapper.message(
        Exception('firebase_auth/invalid-credential'),
      ),
      isNot(contains('firebase_auth')),
    );
  });
}

```

Route integrity workflow

```

# .github/workflows/yohpal_live_ga_gate.yml

name: YohPal Live GA Gate

on:
  pull_request:
  push:
    branches:
      - flutter_app
      - main

jobs:
  release-gate:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: apps/mobile_flutter

    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          channel: stable

```

```
    cache: true

- name: Install dependencies
  run: flutter pub get

- name: Format check
  run: dart format --output=none --set-exit-if-changed lib test

- name: Analyze complete app
  run: flutter analyze

- name: Route integrity
  run: dart run tool/check_routes.dart

- name: Unit and widget tests
  run: flutter test --coverage
```

Required minimum test groups

```
test/features/auth/
test/features/navigation/
test/features/video_feed/
test/features/wallet/
test/features/live_streaming/
test/features/sso/
test/release_gates/
functions/test/gifts/
functions/test/rewarded_ads/
```

Developer Evidence Pack

Developers must return:

1. Git branch
2. Commit hash
3. Crashlytics release-build test crash proof
4. Analytics route fix proof
5. Password reset end-to-end proof
6. Email verification proof
7. Friendly auth error screenshots
8. Environment configuration proof
9. Search navigation proof
10. Live Discovery navigation proof
11. Feed retry and pagination proof
12. Mute and double-tap-like proof
13. Accessibility audit proof
14. LiveKit reconnect proof
15. Duplicate pipeline removal proof
16. Route integrity CI output
17. flutter analyze output
18. flutter test output and coverage
19. Android validation
20. iPhone validation
21. Updated Production Readiness Review

Current Certification Position

Not yet GA-certified.

This code pack closes the correct remaining scope, but certification requires committed code, CI output, device evidence, Crashlytics proof, and a fresh re-audit. The latest verified report still records four Critical items open and the High-severity group essentially unresolved.

IMPLEMENTATION_STATUS_REPORT.pdf

Next Smart Move

Execute **YOHPAL LIVE RELEASE 1.0G — Final Evidence & GA Certification Review** after developers apply Release 1.0F, requiring independent verification of every remaining Critical and High closure, Android/iPhone regression evidence, production observability proof, and the Release Review Board's formal GO, CONDITIONAL GO, or NO-GO decision.